

Logstash 30-Minute Class Exercise

Duration: 30 minutes

Objective: Practice Logstash configuration with input, filter, and output plugins using real-world log data scenarios

Exercise Overview

In this hands-on exercise, you will create Logstash configuration files to process different types of log data. You'll work with **Apache access logs**, apply multiple filter transformations, and output results to both stdout and Elasticsearch.

Prerequisites

- Logstash installed and configured
 - Elasticsearch running on localhost:9200
 - Access to /home/student/elastic-stack/logstash/ directory
 - Basic understanding of Grok, Dissect, Mutate, and CSV filters
-

Sample Data Files

1. Create Apache Access Log File

Create a file at: /home/student/elastic-stack/logstash/input_data/apache_access.log

```
192.168.1.10 - john [03/Nov/2024:14:30:15 +0530] "GET /api/products HTTP/1.1" 200 4523 "https://www.example.com" "Mozilla/5.0"
10.0.0.25 - - [03/Nov/2024:14:31:22 +0530] "POST /api/orders HTTP/1.1" 201 1024 "-" "curl/7.68.0"
172.16.0.5 - mary [03/Nov/2024:14:32:45 +0530] "GET /admin/dashboard HTTP/1.1" 403 512 "https://admin.example.com" "Mozilla/5.0"
192.168.1.10 - john [03/Nov/2024:14:33:10 +0530] "DELETE /api/cart/123 HTTP/1.1" 204 0 "-" "PostmanRuntime/7.29.2"
203.0.113.45 - - [03/Nov/2024:14:35:00 +0530] "GET /images/logo.png HTTP/1.1" 404 256 "https://www.example.com" "Mozilla/5.0"
```

2. Create Employee CSV File

Create a file at: /home/student/elastic-stack/logstash/input_data/employees.csv

```
EmpID,FirstName,LastName,Department,Salary,JoinDate
101, Rahul , Sharma ,Engineering,85000.50,2023-01-15
102,priya,patel,marketing,62000.75,2023-03-20
103,AMIT,KUMAR,sales,55000,2023-05-10
104, Sneha ,Reddy,Engineering,92000.25,2022-11-05
105,vikram,singh,HR,48000,2024-01-08
```

Tasks

Task 1: Parse Apache Access Logs with Grok (10 minutes)

Requirements: 1. Create a configuration file: `apache_grok_exercise.conf` 2. Read from the Apache access log file 3. Use **Grok filter** to parse the log line into these fields: - `client_ip` - `user` (authenticated user) - `timestamp` - `http_method` - `request_path` - `http_version` - `status_code` - `response_bytes` - `referrer` - `user_agent`

1. Use **mutate filter** to:
 - Convert `status_code` to integer
 - Convert `response_bytes` to integer
 - Remove the original message field
2. Output to **stdout** with `rubydebug` codec

Hints: - Apache Combined Log Format pattern: `%{IPORHOST:client_ip} %{USER:ident} %{USER:user} \[%{HTTPDATE:timestamp}\] "%{WORD:http_method} %{URIPATHPARAM:request_path} HTTP/%{NUMBER:http_version}" %{NUMBER:status_code} %{NUMBER:response_bytes} "%{DATA:referrer}" "%{DATA:user_agent}"` - Use `sincedb_path => "/dev/null"` for testing - Set `start_position => "beginning"`

Task 2: Process Employee CSV with Transformations (10 minutes)

Requirements: 1. Create a configuration file: `employee_csv_exercise.conf` 2. Read from the employees CSV file 3. Use **CSV filter** with auto-detect column names 4. Apply **mutate transformations**: - Strip whitespace from `FirstName` and `LastName` - Convert `Department` to uppercase - Convert `Salary` to float - Convert `EmpID` to integer - Rename `FirstName` → `fname` and `LastName` → `lname`

1. Add a **conditional filter**: If `Department` is "ENGINEERING", add a tag `"tech_team"`
2. Output to **both**:
 - `stdout` with `rubydebug` codec

- Elasticsearch at localhost:9200

Hints:

```
if [Department] == "ENGINEERING" {  
  mutate {  
    add_tag => ["tech_team"]  
  }  
}
```

Task 3: Parse Apache Logs with Dissect (Bonus - 10 minutes)

Requirements: 1. Create a configuration file: `apache_dissect_exercise.conf` 2. Use the same Apache access log file 3. Use **Dissect filter** instead of Grok to parse the log 4. Extract the same fields as Task 1 5. Use dissect's `convert_datatype` option to convert: - `status_code` → `int` - `response_bytes` → `int`

1. Add a **conditional output**:

- If `status_code` `>= 400`, output to a file: `/home/student/elastic-stack/logstash/output_data/error_logs.json` with `json_lines` codec
- All logs should also go to `stdout`

Hints: - Dissect pattern for Apache logs:

```
"%{client_ip} %{ident} %{user} [%{timestamp}] \"%{http_method} %  
{request_path} HTTP/%{http_version}\" %{status_code} %  
{response_bytes} \"%{referrer}\" \"%{user_agent}\""
```

Testing Your Configurations

Run each configuration file using:

```
/usr/share/logstash/bin/logstash -f /path/to/your/config_file.conf
```

For Task 1:

```
/usr/share/logstash/bin/logstash -f apache_grok_exercise.conf
```

Verification Checklist

Task 1 Verification:

☐

All 5 log lines are processed

☐

Fields are correctly extracted (check `client_ip`, `status_code`, etc.)

☐

status_code and response_bytes are integers (no quotes in output)

☐

Original message field is removed

Task 2 Verification:

☐

CSV headers are auto-detected

☐

Names are trimmed (no leading/trailing spaces)

☐

Department is uppercase (e.g., "ENGINEERING", "MARKETING")

☐

Salary is float with decimal points

☐

EmpID is integer

☐

Engineering employees have "tech_team" tag

☐

Data appears in Elasticsearch (check with: `curl http://localhost:9200/_cat/indices`)

Task 3 Verification:

☐

Dissect successfully parses Apache logs

☐

Data type conversion works

☐

Error logs (403, 404) are written to error_logs.json file

☐

All logs appear in stdout

Expected Output Sample (Task 1)

```
{
  "@version" => "1",
  "client_ip" => "192.168.1.10",
  "user" => "john",
  "timestamp" => "03/Nov/2024:14:30:15 +0530",
  "http_method" => "GET",
  "request_path" => "/api/products",
  "http_version" => "1.1",
  "status_code" => 200,
  "response_bytes" => 4523,
```

```
"referrer" => "https://www.example.com",  
"user_agent" => "Mozilla/5.0"  
}
```

Troubleshooting Tips

1. **Grok pattern not matching?**
 - Test your pattern at: <https://grokdebugger.com>
 - Check for missing escape characters
 2. **CSV columns not detected?**
 - Ensure first line contains headers
 - Check for proper comma separation
 3. **Logstash won't start?**
 - Check syntax: `/usr/share/logstash/bin/logstash -f your_file.conf --config.test_and_exit`
 - Look for missing braces or quotes
 4. **File not being read?**
 - Verify file path is absolute
 - Check file permissions: `chmod 644 /path/to/file`
 5. **Data not in Elasticsearch?**
 - Confirm Elasticsearch is running: `curl http://localhost:9200`
 - Check index was created: `curl http://localhost:9200/_cat/indices`
-

Bonus Challenge (Extra Credit)

If you complete all tasks early:

1. **Add date filter** to convert Apache timestamp to `@timestamp` field
 2. **Add geoip filter** to enrich `client_ip` with location data
 3. **Create a custom grok pattern** for a different log format
 4. **Use the translate filter** to map status codes to descriptions (200 → “OK”, 404 → “Not Found”)
-

Submission

Save all three configuration files and share: 1. `apache_grok_exercise.conf` 2. `employee_csv_exercise.conf` 3. `apache_dissect_exercise.conf` (bonus)

Test each configuration and take screenshots of successful output.

Learning Outcomes

After completing this exercise, you should be able to:

- ✓ Configure Logstash input plugins (file, stdin)
 - ✓ Apply Grok patterns for complex log parsing
 - ✓ Use Dissect for structured log parsing
 - ✓ Process CSV files with transformations
 - ✓ Apply mutate operations (convert, rename, strip, uppercase)
 - ✓ Implement conditional logic in Logstash
 - ✓ Output to multiple destinations (stdout, Elasticsearch, file)
 - ✓ Debug Logstash configuration issues
-

Good Luck! 🚀